

CS 630: Software Testing and Quality Assurance

9-1 Project Two: Designing a Test and Quality Strategy

Malgorzata Debska

Southern New Hampshire University

Dr. Ben Torrales

September 1, 2026

Quality Strategy and Approach: SquareCalc Home 2.0

This strategy establishes a sustainable quality management system for SquareCalc's Home 2.0 personal-finance product instead of treating quality as a hardening-sprint activity. SquareCalc uses Jira and continuous integration and maintains separate release trains, but it lacks a formal test process, documented user-journey/UAT testing, consistent test-data management, and coordinated hardening testing (SquareCalc Development Process Overview, n.d.). The recommended model is risk-based continuous quality management with shift-left prevention and shift-right production learning. Quality is shared across product, development, QA, DevOps/platform, operations, security, UX, and management, with a quality lead providing governance and objective visibility.

Quality Integration Points Across the Organization

Product management integrates quality during discovery and backlog refinement by defining acceptance criteria, customer value, regulatory constraints, data classification, and measurable nonfunctional requirements. UX contributes to accessibility, error-recovery, and comprehension requirements before implementation. This strengthens SquareCalc's existing Jira-centered product-owner workflow (SquareCalc Development Process Overview, n.d.).

Development integrates quality through coding standards, peer review, unit/component testing, secure coding, static analysis, and API contract tests. QA partners during refinement and implementation, maintain risk/requirements of traceability, perform system and exploratory testing, coordinate UAT, and evaluate release evidence. DevOps/platform engineering integrates quality into CI by executing tests and scans, publishing results, blocking failed mandatory gates,

standardizing environments, versioning dependencies, and providing repeatable synthetic test data. These controls directly address SquareCalc's shared test/stage dependency problems (SquareCalc Development Process Overview, n.d.). Operations and support integrate production quality through monitoring, incident management, customer-impact assessment, and backlog feedback. Logs and telemetry should expose failures and unusual behavior without exposing financial data. Management integrates quality through policy, resources, risk acceptance, and trend review. This documentation is particularly important because most SquareCalc software engineers work remotely (SquareCalc Company Overview, n.d.).

Quality Goals Aligned to Business Priorities and the SDLC

The first goal is customer trust: zero known critical security defects at release and zero unauthorized disclosure of sensitive customer information. The second is transaction correctness: 100% of critical financial rules and state changes must be traceable to requirements and independently verified. The third is reliability: release candidates must meet defined availability, error-rate, and response-time objectives in production-like testing and production. The fourth is release predictability: every release train uses documented entry/exit criteria, named approvers, and a quality dashboard. The fifth is early prevention: reduce escaped Severity 1/2 defects release over release. The sixth is inclusive of usability: critical consumer journeys must satisfy defined accessibility and UAT criteria. The seventh is auditability: requirements, code changes, approvals, tests, defects, and production changes must provide evidence for internal and annual compliance review. These goals reflect product-quality characteristics described by ISO/IEC 25010 (International Organization for Standardization, 2023).

Quality Monitoring and Improvement

The QA lead will track escaped defects, severity, defect removal effectiveness, defect aging, and requirements/risk coverage through Jira dashboards and production-incident links. DevOps will automatically track CI pass rate, flaky tests, build failures, deployment success, and change failure rate from pipeline/deployment telemetry. Operations will track availability, latency, transaction failures, and service-objective performance. Security will track critical/high findings, remediation age, dependency vulnerabilities, privileged-access events, and recurrence. Product/UX will track completion rates, support themes, accessibility defects, and UAT outcomes. Teams review sprint metrics in retrospectives and select concrete improvements; release leadership reviews risk, severe defects, security/performance results, and acceptance evidence before promotion; management reviews trends monthly or by product increment for systemic investment needs. Metrics are used for learning and risk decisions rather than individual performance rankings. Evidence of governance supports monitoring and auditability. Each manual or automated execution should record the linked requirement/risk, exact URL or endpoint, environment/build, browser/version or device, viewport where applicable, timestamp, test data/preconditions, reproduction steps, expected/actual results, status, and screenshot/log/API/CI evidence. Defects and release decisions link back to that record. This makes quality conclusions reproducible and supports SquareCalc's annual compliance obligations, including PCI expectations (SquareCalc Development Process Overview, n.d.; PCI Security Standards Council, 2024).

Quality Plan Approach and Justification

The chosen approach is risk-based continuous quality management with shift-left prevention and shift-right production learning. It was developed by mapping SquareCalc's actual process—Jira requirements, developer-led testing, CI, shared environments, inconsistent data, hardening

sprints, rapid API fixes, release trains, and annual audits—to the points where quality risk enters or escapes (SquareCalc Development Process Overview, n.d.). Controls are placed at the earliest practical point: synthetic data governance before system testing, contract tests and version controls in CI, explicit release of exit criteria, and production feedback that creates regression tests or corrective actions. This approach is preferable to a traditional test-last model in which a separate QA team receives completed software near release. Test-last would reinforce SquareCalc's current hardening problem, where testing competes with schedules, unstable environments, and release deadlines. It also cannot prevent unclear requirements, insecure design, weak test data, or operational gaps. Continuous quality preserves independent QA judgment while distributing prevention across the teams that create requirements, code, infrastructure, releases, and customer support. The approach also supports compliance without reducing quality to an annual audit exercise. SquareCalc's normal work should produce traceable approvals, controlled access, test evidence, change records, and repeatable release gates. PCI DSS similarly emphasizes defined controls and testing around systems handling payment data (PCI Security Standards Council, 2024). Any exception to a quality control should identify the risk, compensating control, owner, expiration date, and management approval.

Quality Strategy and Development Process

The target process begins with discovery and risk classification. During refinement, product, UX, development, QA, and security define acceptance criteria and nonfunctional expectations. During implementation, developers use peer review and automated unit/component/API tests while CI performs required scans and checks. QA conducts risk-based systems, integration, exploratory, accessibility, and end-to-end validation with controlled synthetic data. Stage receives a versioned release candidate for production-like performance, security, and UAT

evidence; release leadership then reviews the dashboard and documented exceptions before deployment. After deployment, operations monitor service objectives and customer-impact signals while supporting product feed incidents into Jira. Severe escapes trigger root-cause analysis focused on both the defect and the failed prevention/detection mechanism. Corrective actions may add tests, improve acceptance criteria, strengthen monitoring, change environment controls, or require training or architecture work. Retrospectives and product-increment reviews determine whether those actions improved the metrics, creating a closed loop of defining quality, building it in, verifying it, observing it, learning, and improving.

References:

SquareCalc Company Overview. (n.d.). CS 630 Project Two course supporting material. Southern New Hampshire University.

SquareCalc Development Process Overview. (n.d.). CS 630 Project Two course supporting material. Southern New Hampshire University.

International Organization for Standardization. (2023). ISO/IEC 25010: Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model.

PCI Security Standards Council. (2024). Payment Card Industry Data Security Standard (PCI DSS), Version 4.0.1.